



GT Influencer Project

IP Crowd VIP

What is PNAMBIC?



→ Pay No Attention to the Man Behind The Curtain

→ Implementations are not complete, but serve as foundation for later semesters

→ Some features are closer to being done than others, but none are ready for production

Problem

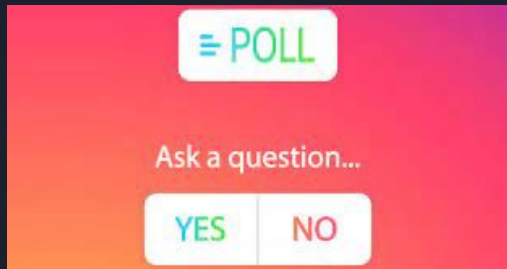


Individuals with large social media followings, known as influencers, are able to ask their audiences for advice on day-to-day issues.



How can we mimic this experience within students who do not have the same status?

Solution?



Polls!

- Users can ask and answer questions in a polling format
- Favorited/common answers will bubble to the top
- Through gamification, TLDR generator, and automated hashtags, the app is easy and fun to use.

#hashtag

- Automated Hashtag Generator based on inputted text

SUMMARY

- Short text summary based on question



- Gamification based on answer credibility



- UI connecting back-end to front-end

Hashtag Generation



- Hashtags attract more people
 - Better hashtags provide better exposure to the public
 - Keywords of the user's post should be extracted
- TF-IDF(Term Frequency - Inverse Document Frequency)
 - TF: how **often** does the word appear in a **sentence**?
 - IDF: how **rare** is the word in the **database**?
 - Multiply TF and IDF to assign scores to all words

```
texts.csv X
IPCrowd > Backend > texts.csv
1 user_input_sentence
2 There was no telling what thoughts would come from the machine.
3 "Cats are good pets, for they are clean and are not noisy."
4 Despite multiple complications and her near-death experience
5 "I'm not a party animal, but I do like animal parties."
6 That must be the tenth time I've been arrested for selling deep-fried cigars.
7 He learned the important lesson that a picnic at the beach on a windy day is a bad idea.
8 Swim at your own risk was taken as a challenge for the group of Kansas City college students.
9 "It was obvious she was hot, sweaty, and tired."
10 The door slammed on the watermelon.
11 She thought there'd be sufficient time if she hid her watch.
12 The fence was confused about whether it was supposed to keep things in or keep things out.
13 Two seats were vacant.
14 The pigs were insulted that they were named hamburgers.
15 She finally understood that grief was her love with no place for it to go.
16 "There have been days when I wished to be separated from my body, but today wasn't one of those days."
17 There aren't enough towels in the world to stop the sewage flowing from his mouth.
18 "Despite what your teacher may have told you, there is a wrong way to wield a lasso."
19 "She wrote him a long letter, but he didn't read it."
20 The teens wondered what was kept in the red shed on the far edge of the school grounds.
21 Sarah ran from the serial killer holding a jug of milk.
22 They ran around the corner to find that they had traveled back in time.
23 The sudden rainstorm washed crocodiles into the ocean.
24 Love is not like pizza.
25 While on the first date he accidentally hit his head on the beam.
26 I love eating toasted cheese and tuna sandwiches.
27 She was too short to see over the fence.
28 There's a reason that roses have thorns.
29 Every manager should be able to recite at least ten nursery rhymes backward.
30 The changing of down comforters to cotton bedspreads always meant the squirrels had returned.
31 "There was no ice cream in the freezer, nor did they have money to go to the store."
32 Eating eggs on Thursday for choir practice was recommended.
```



TF-IDF

- Term Frequency (TF)
 - Counting the frequency of a certain word in the user's sentence
 - The more it appears, the more important the term is
- Inverse Document Frequency (IDF)
 - Counting the frequency of a certain word in the database
 - The less it appears, the more important the term is for its sentence
 - Repeated words (the, his, her, etc.) are ignored

I love soccer. I want to play soccer.

I like basketball. I want to sleep.

I am a student at Georgia Tech.

I need to eat something!

I want to go back home :(



TF-IDF

- Term Frequency (TF)
 - Counting the frequency of a certain word in the user's sentence
 - The more it appears, the more important the term is
- Inverse Document Frequency (IDF)
 - Counting the frequency of a certain word in the database
 - The less it appears, the more important the term is for its sentence
 - Repeated words (the, his, her, etc.) are ignored

I love soccer. I want to play soccer.

$$I = 2 * (1)^{-5}$$
$$\text{soccer} = 2 * (1)^0$$

I like basketball. I want to sleep.

I am a student at Georgia Tech.

I need to eat something!

I want to go back home :(

TF-IDF

- Term Frequency (TF)
 - Counting the frequency of a certain word in the user's sentence
 - The more it appears, the more important the term is
- Inverse Document Frequency (IDF)
 - Counting the frequency of a certain word in the database
 - The less it appears, the more important the term is for its sentence
 - Repeated words (the, his, her, etc.) are ignored

```
import pandas as pd
from scipy import rand
from sklearn.feature_extraction.text import TfidfVectorizer
from data_preprocessing import clean_text, get_stopwords
from tf_idf import get_hashtags
from enter_sentence import add_new_sentence
```

```
def extract_num_from_vector(feature_names, sorted_items, top_num = 5):
    """get feature names and tf-idf scores of top 5 items"""

    # only use top 5 items from vector
    sorted_items = sorted_items[:top_num]

    scores = []
    features = []

    for index, score in sorted_items:
        scores.append(round(score, 3))
        features.append(feature_names[index])

    result = {}
    for idx in range(len(features)):
        result[features[idx]] = scores[idx]

    return result
```

```
def get_hashtags(vectorizer, feature_names, inputs):
    """Return hashtags from user input"""

    # generate tf-idf for the user input
    tf_idf_vector = vectorizer.transform([inputs])

    # sort the tf-idf vectors in decreasing order of scores
    sorted_items = sort_matrix(tf_idf_vector.tocoo())

    # generate hashtags
    hashtags = extract_num_from_vector(feature_names, sorted_items, 5)
```

Data Preprocessing

- Stopwords
 - Transition words
 - Conjunction words
- Punctuations, whitespaces, and null types
 - .,/!@#\$%^n/a

```
import re

def get_stopwords(file_path):
    # loading stopwords from the stopwords.txt file

    with open(file_path, 'r', encoding = "cp1252") as f:
        stopwords = f.readlines()
        stop_set = set(m.strip() for m in stopwords)

        return list(frozenset(stop_set))

def clean_text(text):
    # need to clean the existing texts for better generation

    # lowercase them
    text = text.lower()

    # remove punctuations
    punctuations = "'\"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~""
    text = "".join([c for c in text if c not in punctuations])

    # remove whitespaces and new lines
    text = re.sub('\s', ' ', text)

    return text
```



Perform Demo



Features to Implement

Create a user suggestion system

- Allows users to enter hashtags they thought should have been generated
- If many users suggest the same hashtags, it appears in the auto-generation
 - User enters "I love Georgia Tech!" → #YellowJackets or #Atlanta appears as well

Text Summarization Demo

- Summarizing responses accurately is crucial to creating TLDRs
- Solution
 - Split response into words and sentences
 - Assign a score to each sentence based on its frequency of overall words
 - Add all sentence with greater than expected scores (usually 1.1x) to the summary
- Sentence approach prevents false statements from the summarizer

```
text = "According to all known laws of aviation, there is no way a bee should be able to fly. Its wings are too small to get its fat little body off the ground. The bee, of course, flies anyway because bees don't care what humans think is impossible. Yellow, black. Yellow, black. Yellow, black. Yellow, black. Ooh, black and yellow! Let's shake it up a little. Barry! Breakfast is ready! Ooming! Hang on a second. Hello? - Barry? - Adam? - Can you believe this is happening? - I can't. I'll pick you up. Looking sharp. Use the stairs. Your father paid good money for those. Sorry. I's excited. Here's the graduate. We're very proud of you, son. A perfect report card, all B's. Very proud. Ma! I got a thing going here. - You got lint on your fuzz. - Oa! That's me! - Wave to us! We'll be in row 118,000. - Bye! Barry, I told you, stop flying in the house! - Hey, Adam. - Hey, Barry. - Is that fuzz gel? - A little. Special day, graduation. Never thought I'd make it. Three days grade school, three days high school. Those were awkward. Three days college. I'm glad I took a day and hitchhiked around the hive. You did come back different. - Hi, Barry. - Artie, growing a mustache? Looks good. - Hear about Frankie? - Yeah. - You going to the funeral? - No, I'm not going. Everybody knows, sting someone, you die. Don't waste it on a squirrel. Such a hothead. I guess he could have just gotten out of the way. I love this incorporating an amusement park into our day. That's why we don't need vacations. Boy, quite a bit of pomp... under the circumstances. - Well, Adam, today we are men. - We are! - Bee-men. - Asen! Hallelujah! Students, faculty, distinguished bees, please welcome Dean Buzzwell. Welcome, New Hive City graduating class of... ..9:15. That concludes our ceremonies. And begins your career at Honex Industries! Will we pick our job today? I heard it's just orientation. Heads up! Here we go. Keep your hands and antennas inside the tram at all times. - Wonder what it'll be like? - A little scary. Welcome to Honex, a division of Honesco and a part of the Hexagon Group. This is it! Now. Wow. We know that you, as a bee, have worked your whole life to get to the point where you can work for your whole life. Honey begins when our valiant Pollen Jocks bring the nectar to the hive. Our top-secret formula is automatically color-corrected, scent-adjusted and bubble-contoured into this soothing sweet syrup with its distinctive golden glow you know as... Honey! - That girl was hot. - She's my cousin! - She is? - Yes, we're all cousins. - Right. You're right. - At Honex, we constantly strive to improve every aspect of bee existence. These bees are stress-testing a new helmet technology."
```

```
According to all known laws of aviation, there is no way a bee should be able to fly. Its wings are too small to get its fat little body off the ground. The bee, of course, flies anyway because bees don't care what humans think is impossible. Yellow, black. Yellow, black. Yellow, black. Yellow, black. - Hi, Barry. Boy, quite a bit of pomp... under the circumstances. - Well, Adam, today we are men. Students, faculty, distinguished bees, please welcome Dean Buzzwell. Welcome, New Hive City graduating class of... ..9:15. Keep your hands and antennas inside the tram at all times. Welcome to Honex, a division of Honesco and a part of the Hexagon Group. We know that you, as a bee, have worked your whole life to get to the point where you can work for your whole life. Our top-secret formula is automatically color-corrected, scent-adjusted and bubble-contoured into this soothing sweet syrup with its distinctive golden glow you know as... Honey! - Yes, we're all cousins. - At Honex, we constantly strive to improve every aspect of bee existence. These bees are stress-testing a new helmet technology."
```



Answer credibility

Credibility labeling:

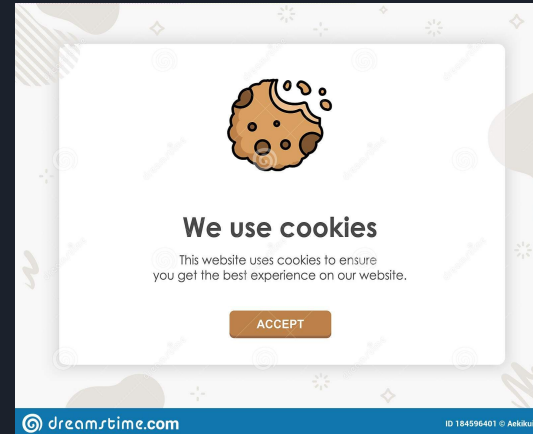
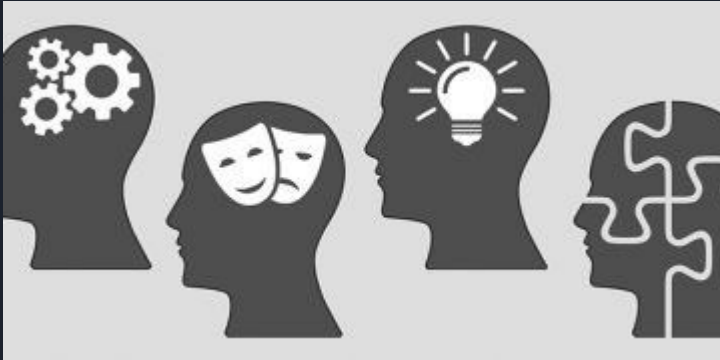
We can base our credibility labeling on 3 variables:

- Authority:
 - Follower score (number of followers the author has)
 - Mention score (mentions to the author)
 - List score (users that see the answer)
 - Popularity score (retweets)
- Currency: time measuring. Old or new messages, interactions.
- Relevance: with the question and with the user asking (personality test)

Answer credibility: Personality tests

Knowing what are the hobbies, favorite clothes, food, brands, sports or entertainments can really help to classify information. Credibility can also be classified taking this statement into account.

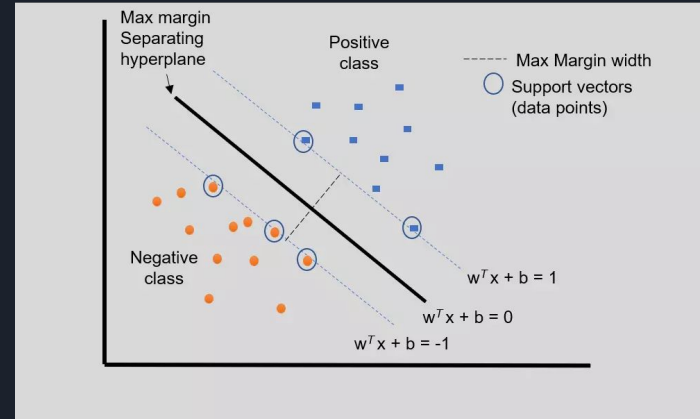
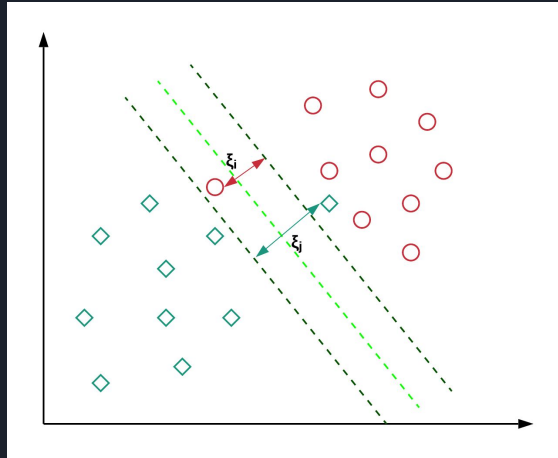
Instagram, twitter or facebook base their algorithms in cookies and tracking your information so that they can show what you really like to see in their apps. Personality tests are much more private and only focused on bias information.



Answer classification and SVM (support vector machine)

Classification:

- Definitely credible
- Seems credible
- Definitely non-credible
- Related to topic, no information
- Unrelated to topic



Answer credibility demo

The confidence sort uses Wilson score interval and the mathematical notation looks like this:

$$\frac{\hat{p} + \frac{1}{2n}z_{1-\alpha/2}^2 \pm z_{1-\alpha/2}\sqrt{\frac{\hat{p}(1-\hat{p})}{n} + \frac{z_{1-\alpha/2}^2}{4n^2}}}{1 + \frac{1}{n}z_{1-\alpha/2}^2}$$

In the above formula the parameters are defined in a following way:

- p is the observed fraction of positive ratings
- n is the total number of ratings
- $z_{\alpha/2}$ is the $(1-\alpha/2)$ quantile of the standard normal distribution

```
from math import sqrt

def _confidence(ups, downs):
    n = ups + downs

    if n == 0:
        return 0

    z = 1.281551565545
    p = float(ups) / n

    left = p + 1/(2*n)*z*z
    right = z*sqrt(p*(1-p)/n + z*z/(4*n*n))
    under = 1+1/n*z*z

    return (left - right) / under

def confidence(ups, downs):
    if ups + downs == 0:
        print(0)
        return 0
    else:
        return _confidence(ups, downs)

print(confidence(0,10))
```



Answer credibility demo

However, up vote and down vote is not everything when it comes to answer credibility....

Let's look at the user history and the answer itself.

"I don't really know, but just do what you want."

=> num_punctuation = 2, num_words = 10, num_characters = 34

"According to one study, studying in the morning gives better results than staying up all night. In fact, you are way more focused."

=> num_punctuation = 4, num_words = 23, num_characters = 108

num_followers, num_friends, num_statuses, account_age

1.265808823529412
2.4009609500805156

```
#account history is a sign of credibility
def credibility_user(num_followers, num_friends, num_statuses, account_age):
    return num_followers/account_age + num_friends/account_age + num_statuses/account_age

#bad grammar is a sign of non-credibility
def credibility_status(num_punctuation, num_words, num_characters):
    return num_punctuation/num_characters + num_punctuation/num_words
```

Gamification

Encourage people to work and be interested in this platform:

- Verified people (answer credibility reward)
- Time measured. Lock important answers for days (reduce algorithm work). Not only responding good info, but being constant
- Preference for platinum people:
 - Constantly active
 - Verified
 - Number of positive interactions
 - Followers



Platinum people earn popularity → gives them the opportunity to promote brands → earn money





Gamification Data - Psychology of User

- Current .csv file has only certain iterations of hashtags for certain sentences.
- Users can be afforded the ability to provide 'suggestions' for hashtags, which can be filtered through and added to the existing .csv file
- This can improve the accuracy and precision of the algorithm for generation down the line.



Future Plans, Further Implementations

- Gather more data for hashtag generation
- Optimize the answer credibility algorithm
- Consolidate all algorithms into one larger program.
- Create a robust front-end

A blue parallelogram and a light green parallelogram are positioned in the top-left corner of the slide. The blue shape is partially behind the green one. Both shapes are oriented diagonally, with their longer sides running from the top-left towards the bottom-right. The background of the entire slide is a dark navy blue, featuring faint, larger-scale diagonal stripes in a slightly lighter shade of navy.

Questions?